

> append: **a.append()** add an element in the end of list

> concatenate: **a+b**, join two or more lists

> append is same as **std::vector**, pre-allocate space, fast; **concat** is slow.

> cannot delete iteratively, since the **next()** will be also deleted

> 当创建一个 list 并赋值给一个变量时,该变量实际上是一个指向对象的引用。这意味着,当在函数中修改一个可变对象 (list, dict...) 时,你实际上是修改了该对象的内容,而不是引用。也就是说,函数内可以修改函数外的 list。

> **insert()**, **pop()**, **remove()**, **in**, **index()** with **O(1)**, others **O(n)**

List Comprehension: **list = [i for i in range(1,101)]**

Generator Expression:

> **list_gen = (i for i in range(1,101))**

> returns an iterator, generate element in demand.

> requires less memory

Tuple:

> Immutable, cannot be modified, static array.

> Can contain one or more types in a list, defined using (**)**

> With only one element: **tuple1 = (3,)**

> List usually stores a large collection of data with the **same type**

> Tuple usually stores a small collections of items with **various types**

> **list()**: Change tuple to list: **tuple1()**: change list to tuple

> **len()** **has O(1)**, others **has O(n)**

Set:

> unordered, mutable, no duplicate elements

> only **len(a)**, **min(a)**, **max(a)**, **x in a**

>>> **setA = {1,2,3,4}**

>>> **setB = {3,4,5,6}**

>>> **setA | setB** **#Union**

{1,2,3,4,5,6}

>>> **setA & setB** **#Intersection**

{3,4}

>>> **setA - setB** **#A-B**

{1,2}

>>> **setA ^ setB** **#(A|B)-A&B (Symmetric Difference)**

{1,2,5,6}

> **add()**, add single element. **update()**, add multiple elements.

> **delete()**, **discard()** remove element, delete will throw error if element is not exist.

> **pop()** delete and return an element

> **clear()** delete all elements

> **set()**, **list()** for list and set **ONLY**

> 所有操作都是 **O(1)**

Dictionary:

> search key in the dict: **dict.get("apples")** or **dict["apples"]**.

each key has a corresponding value.

> 使用 **dict.get()** 时, 如果键不存在会返回 **None**. 使用 **dict[key]** 时会返回 **KeyError**

> each pair has a key(left) and a value(right)

> can store any type

> Delete: **dict.pop("apples")** or **del dict["apples"]**

> **clear()**: clear all. **copy()**: make a copy. **keys()**: return all keys.

> **values()**: return all values. **items()**: return all keys + values

> 不能修改正在迭代的容器的大小或结构, 但可以改变键的值

> 如果在加入键值对时, 键已经存在, 那么其键对应的值会被覆盖为新值

> **dict[key].append()** is valid

> **dict.keys()**, **values()**, **items()** 是 **O(n)**, 其余操作都是 **O(1)**

Lambda:

>>> **(lambda x:x)(10)** **# Identity function**

10

>>> **(lambda x: 'abc')(5)** **# Constant function**

'abc'

>>> **(lambda x,y,z: x*y*z)(4,5,9)** **# Multiple arguments**

18

>>> **def func_a(n)**

>>> **return lambda x:x+n**

>>> **f1 = func_a(10)**

>>> **f1(1)**

11

Variable store a function:

> **say_hello = greet()**, store the output.

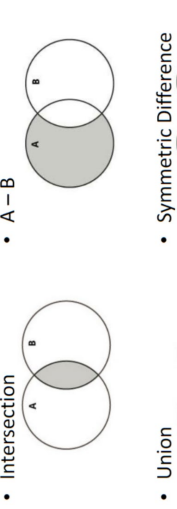
> **say_hello = greet**, store the function

> Their id are identical

> function can store in list, tuple, set, dict

> function can be passed as argument to functions

> function can be returned from function



Closures:

> remembers the state and the environment

> returns an inner function

> preserve function state across function calls

Decorators:

> all decorators are closures

> for decorators, the outer function accepts a function as input arg

Pastpaper Question

> **int('12.210')** throws an error, cannot be string

> **['a', 'b', 'c', 'd'][::-1]** 即倒序排列, 即 **['d', 'c', 'b', 'a']**

> **['a', 'b', 'c', 'd'][-1]** = **['d']**

> **['a', 'b', 'c', 'd'][:1]** = **['a']**

> **['a', 'b', 'c', 'd'][:1:-1]** = **['b', 'c']**

> **list.append('IT5001')** 不打印任何结果, 应该选 none

> **3 in [1,2,{3,4}]** 此 set 包含了一个 set, 而 set 中的元素必须是可散列的 (hashable), 然而 set 本身是不可散列的, 所以此 set 是非法的, 同理, tuple 中包含 tuple 也是非法的

> **lambda x: x (3)(lambda x: x*4)** 前面一个 lambda 是带常数的, 并且没有附加计算, 所以为 3. 第一个 lambda 的输出可以看作是第二个 lambda 的输入. 即这个表达式可以写为 **lambda x: x*4 (3)**. 所以输出为 12

> **1(2+3)%4** 此表达式中的 1 会被 python 理解为函数名称, 所以会抛出错误

> **[1, 2] + (3, 4)** list and tuple 不能相加, 因为是不同的数据结构, 可以将 list 或 tuple 转换成对方的结构后相加.

> **x = [5, 0, 0, 1] += 'IT'** string 是一个可以被迭代的, 所以 IT 会被拆分为 I 和 T. 输出为 **[5, 0, 0, 1, 'I', 'T']**

> **[1, 2, 3][4:5]** 和 **IT5001!** 其中, **[1, 2, 3][4:5]** 会返回一个空列表, 并会返回第一个逻辑为 false 的值, 如果所有制御都为 True, 则返回最后一个值. 空列表被看作作是 False, 而任何非空的对象都会是 True. 所以此表达式会返回空列表 []

> **list(filter(bool, [0, 1, 2]))** bool 会测试每个元素的布尔值, 然后 filter 只会保留布尔值为 True 的值, 所以输出为 **[1,2]**

Built-in Functions

> **round()** 舍入到最近的偶数, **round(2.5)=2, round(3.5)=4**

> **input()** 函数总是返回一个字符串类型的值

> **split()** 按照指定的分隔符将字符串拆分成一个子字符串列表, 如没有指定分隔符, 那么默认是空白字符, 如空格, 换行符, 制表符. **split('w')**

> **strip()** 移除开头和结尾的指定字符, 若未指定, 则移除空白字符. 还可分为 **lstrip()** 移除开头的指定字符, **rstrip()** 移除末尾指定字符

逻辑运算与计算

> 逻辑运算符 (如 or) 是短路求值的. 这意味着如果左边的操作数已经确定了整个表达式的值, 那么右边的操作数就不会被评估.

例如 **1+1+1 or 3>7-6 or 6>7/0**, 第二个操作数为 True, 那么 python 不会计算之后的操作数, 所以不会识别到 **ZeroDivisionError**:

> 逻辑运算符中, and 的运算顺序要优于 or

> **11&4**: 这个表达式使用了按位与运算符 & 来计算两个整数 11 和 4 的二进制按位与结果. **1011 & 0100 = 0000 = 0**

> 在多重比较时, 会从左到右进行评估. 例如 **False == True == False**, 其实是 **False == True and True == False**

> 在多重比较时, 会从左到右进行评估. 例如 **False == True == False**, 其实是 **False == True and True == False**

Break, continue, pass

> **break** 用于完全退出循环. **continue** 用于跳过当前迭代并继续下一个循环. **pass** 是一个空操作, 仅作为占位符

map() & filter()

> **map** and **filter** 函数都返回一个可迭代的对象, 只能使取一次

Lambda Function

> Lambda 函数的输入如果在 for loop 中, 那么他只会循环 for loop 的最终值 例如 **lambda x: i + x for i in range(3)**, 此处的 i 只会是 3

OOOP

> super() 函数在 Python 中用于调用父类 (或超类) 的方法. 用法: **super().function(arg)**

> 当一个类使用了多重继承时 **class sub3(sub1, sub2)**, 那么 **super()** 函数会按照顺序依次调用父类的对应函数, 顺序为 **sub1, sub1 的父类, sub1 父类的父类...**, **sub2, sub2 的父类...** 若 **sub1** 和 **sub2** 共享一个父类, 那么 **sub1** 之后为 **sub2**

> **__function_name()** is a private member function, use **object.function_name()** to access

成员函数:

> 子类会自动继承父类的所有成员函数. 如果子类重定义了某个成员函数, 那么该成员函数在子类中的版本会覆盖父类中的版本. 子类可以通过 **super()** 函数来调用父类的成员函数.

构造函数 (init_):

> 如果子类没有定义自己的构造函数, 那么它会自动继承父类的构造函数.

> 如果子类定义了构造函数, 那么它需要明确地调用父类的构造函数 (如果希望执行父类的构造操作). 这通常通过 **super().__init__()** 来实现.

> 如果子类的构造函数没有调用父类的构造函数, 那么父类的构造函数不会自动执行.

成员变量:

> 子类会“继承”父类构造函数中初始化的成员变量的值. 但是, 如果子类的构造函数调用了父类的构造函数, 那么父类的构造函数会执行, 从而初始化其成员变量. 如果子类和父类具有相同名称的成员变量, 并且子类的构造函数没有调用父类的构造函数, 那么子类的成员变量会覆盖父类的成员变量.

Read File

> **b** for binary format. all mode with **b** open the file in binary format

> **r** for reading and writing. all mode with **+** will open the file with **R/W**

> **r** for read only. all mode with **r** has file ptr at the beginning of the file

> **w** for write only. overwrite the file if exists, create file otherwise. File ptr at the beginning of the file

> **a** for appending. File ptr at the end. Create new file if not exist

> **r**, **r+**, **rb+**, **w**, **wb**, **wt**, **wb+**, **a**, **ab**, **ab+**

Deep Count

Deep Count

```
1 def deepcount(seq):
2     if seq == []:
3         return 0
4     elif type(seq) != list:
5         return 1
6     else:
7         return deepcount(seq[0]) + deepcount(seq[1:])
```

Deep Map

```
1 def deepMap(func, seq):
2     if seq == []:
3         return seq
4     elif type(seq) != list:
5         return func(seq)
6     else:
7         return [deepSquare(func, seq[0])] + deepSquare(func, seq[1:])
```

Change List to tuple deeply

```
1 def deep_tuple(s):
2     if not isinstance(s, list):
3         return tuple(deep_tuple(i) for i in s)
```

Flatten

```
1 def flatten(seq):
2     if seq == []:
3         return seq
4     elif type(seq) != list:
5         return [seq]
6     else:
7         return flatten(seq[0]) + flatten(seq[1:])
```

dict: d[a] if no a: **KeyError**,
d.get(a): None or default
list: l.find(a) if no a: -,
l.index(a): **Value Error**

$\neq \bar{\tau}$ Class (eg. class Duck)

- instance 不是 dict: Duck()['sound'] → TypeError
- 子承母类, 子是否有某属性取决于是否有重写 init
- AttributeError 比如:

```
class A:
    def __init__(self, x):
        self.x = x
    def f32(self.a):
        self.x += a.x

class B(A):
    def __init__(self, y):
        self.y = y
```

- 一旦定好了 x , 后续无法修改
比如 $Alan = A(Alan)$, 若执行 $Alan.name = Alan$, 不会改变 $Alan$

• $\text{alan} = D()$
 $\text{class } A \dots$
 $\text{class } B(A) \dots$
 $\text{class } C(A) \dots$
 $\text{class } D(B, C)$

若 alan.say , 则先 $\rightarrow B \rightarrow B$ 的 super 到 $C \rightarrow C$ 的 super 到 B 的 super
 $\rightarrow A \rightarrow C$ 的类系

0 and 1 = 0, " and 1 = ", 1 or 1/0 = 1
 A and B gives A if A is falsey or B if B is truthy
 A or B gives A if A is truthy or B if B is falsey
 not A gives True if A is falsey or False if A is truthy

Higher Order Function
• integration approximation $\rightarrow$ Newton's method.

```
def deriv(f):
    dx = 0.000000001
    return lambda x: (f(x+dx)-f(x))/dx

>>> def f(x):
    return x**3+3*x-1

>>> deriv(f)(9)
246.000001324870388
```

elif type(seq) == list: return seq
 else: return f(seq[:2]) + f(seq[2:])
 (s = f(1,2,3,4,5,6,7))
 s = [7,6,5,4,3,2,1]

• Python 运算顺序
 () > 算术 > 逻辑 (not > and > or)
 and 和 & or 和 |
 点乘 * 点除 /

T F A
T T A
F T B
F F B

T T B
T T A
F F A
F F A

- & ($3 > 2$) & ($4 > 5$) False
set: {1, 2, 3} & {1, 2, 3, 4} → {}
 $5 \& 3 : 5$ 的 = 二进制 101
3的 = 二进制 011
3的 = 控制位
 $\begin{matrix} 5: 101 \\ 3: 011 \\ \hline 7: 111 \end{matrix}$
 $5 \times 10^2 + 0 \times 10^1 + 1 \times 10^0 = 1$
 $5 \& 3 = 1$

关于 deep_tuple(s)

- def d_t(s):
if not isinstance(s, list):
return s
return tuple(d_t(i) for i in s)

```

class Character(object):
    def __init__(self):
        self.name = 'None'
        self.maxhp = 1000
        self.hp = 1000
        self.str = 0
        self.manna = 0
        self.mana = 0
        self.cost = 9999999999
        self.alive = True

    def act(self, myTeam, enemy):
        pass

    def gothurt(self, damage):
        if damage > self.hp:
            self.hp = 0
            self.alive = False
        else:
            self.hp -= damage

class Fighter(Character):
    def __init__(self):
        super().__init__()
        self.name = 'Fighter'
        self.maxhp = 1200
        self.hp = 1200
        self.str = 100
        self.cost = 100

    def act(self, myTeam, enemy):
        target = randallive(enemy)
        enemy.target().gothurt(self.str)

class Radeem(Fighter):
    def __init__(self):
        super().__init__()
        self.name = 'Radeem'
        self.cost = 200
        self.hp = self.maxhp = 900
        self.power = 0

    def gothurt(self, damage):
        super().gothurt(damage // 2)
        self.power = damage // 4

    def act(self, myTeam, enemy):
        if self.power >= 500:
            for i in range(1):
                super().str(myTeam, enemy)
                self.power = 0
        else:
            super().act(myTeam, enemy)
            self.power = 100

```

无子又付的开心的

模式	读写权限	文件存在时	文件不存在时	指针初始位置	典型用途
r	只读	正常打开	报错	开头	读文本文件
w	只写	清空内容	创建文件	开头	覆盖写文件
a	追加	追加到末尾	创建文件	末尾	日志追加
x	只写	报错	创建文件	开头	独占式创建文件
rw	读写	正常打开	报错	开头	读写已有文件
aw	读写	清空内容	创建文件	开头	新建并覆盖写文件
ax	读写	追加到末尾	创建文件	末尾	追加并覆盖写文本文件
rwx	二进制造型	同上列二进制式	同上列二进制式	同上列	处理图片/音频等二进制文件

- 关于 Image 处理
 - load image, package 'imageio'
 - pip 用于 download imageio or numpy, pandas 等 python 第三方包仓库
 - Broadcasting - numpy
 $\text{cat_pic2} = \text{cat_pic} * [0, 1, 0]$
 - numpy 除了 Broadcasting, 还可以 Fourier Transform, Uniform filter; Histogram; Laplace; Making Mask; Negative Image
 - PILLOW
 PIL $\rightarrow$ python Imaging Library
 - exit (exchangeable image file format): metadata

• $\text{sum}(\text{map}(\lambda x: x(2), [\lambda x: i+x \text{ for } i \text{ in } \text{range}(3)]))$
 more of digits in: $\lambda x: i+x$ for i in $\text{range}(3)$ has effect.
 $\text{def} \text{ add}(a, b): \text{str}(a+b)$
 $\text{add}(7: \text{int}(x) \neq 2, \text{str}(n), 0)$
 $\lambda x: 2+x, \lambda x: 2+x, \lambda x: 2+x$
 to ensure 1-digit case $(\lambda x: y(x(2))) (\lambda x: x(1))$
 $\lambda y: y(2) (2) = 2 * 2 * 2 = 8$
 $\Rightarrow \text{sum}([4, 4, 4]) = 12$

Final 错题

```
def deep_reverse(seq):  
    if type(seq) != list or not seq:  
        return seq  
  
    return deep_reverse(seq[1:]) + [deep_reverse(seq[0])]  
  
[1, 2, [3, 4, [5, 6, [7], 8], 9], 10, [11]]  
nt(deep_reverse(l))  
[1, 10, 9, 8, 7, 6, 5, 4, 3, 2, 11]
```

```
= [1: 2, 2: 4, 3: 6, 4: 7]
r k in a:
    if k%2 == 1:
        del a[k]
print(a)
```

import matplotlib.pyplot as plt

plt.plot:

plt.bar:

(x-axis: dots points)

plt.hist: x-axes ranges

plt.pie:

[a[i]] ← dict 被修改, 无法继续 iteration

> Runtime Error.

```
s = 0
= [1, 2, 3, 4, 5]
r i in l:
    l.pop(0)
    ans += i
print(ans)
```

Operator Precedence:

()	[]	+X	%	.	:	in	not
*	/	-	^	<	>	is	and
+	+	<<	>>	==	!=	is not	.or.

又 $\text{pop}()$ 后, $l = [2, 3, 4, 5]$,
个循环会从 $\text{index} 0$ 到 $\text{index} l$ 在新里
 $\text{pi} = 3$.

```

bda x: (lambda f: f(f)) (lambda y: x(lambda z: y(y)(z)))
bda a: lambda b: lambda c: Z(lambda f: lambda x: a if b(x) else c(f, x))

factorial function as a single-line expression making use of  $\lambda$ :

a1 = r(1) (lambda x: x == 0) (lambda f, x: x * f(x-1))

else:
    return deep_sum(seq[0]) + deep_sum(seq[1:])

def deep_count(seq):
    if len(seq) == 0:
        return 0
    else:
        return 1 + deep_count(seq[1:])

def deep_max(seq):
    if len(seq) == 0:
        return None
    else:
        return max(seq[0], deep_max(seq[1:]))

def deep_min(seq):
    if len(seq) == 0:
        return None
    else:
        return min(seq[0], deep_min(seq[1:]))

def deep_max_min(seq):
    if len(seq) == 0:
        return None
    else:
        return min(deep_min(seq[0]), deep_min(seq[1:]))
        return max(deep_max(seq[0]), deep_max(seq[1:]))

def deep_flatten(seq):
    if len(seq) == 0:
        return []
    else:
        return deep_flatten(seq[0]) + deep_flatten(seq[1:])

def deep_reverse(seq):
    if len(seq) == 0:
        return []
    else:
        return deep_reverse(seq[1:]) + [deep_reverse(seq[0])]

def deep_search(seq, target):
    if len(seq) == 0:
        return False
    else:
        return target == seq[0] or deep_search(seq[1:], target)

def deep_any(seq, pred):
    if len(seq) == 0:
        return False
    else:
        return pred(seq[0]) or deep_any(seq[1:], pred)

def deep_all(seq, pred):
    if len(seq) == 0:
        return True
    else:
        return pred(seq[0]) and deep_all(seq[1:], pred)

def deep_map(seq, func):
    if len(seq) == 0:
        return []
    else:
        return [func(seq[0])] + deep_map(seq[1:], func)

```